Data Analytics Project: IGN Game Reviews Analysis

Dataset Used: ign.csv

This document outlines a structured, end-to-end data analysis project based on game reviews from the IGN dataset. The project is divided into five scenarios, increasing in difficulty from basic data handling to advanced analysis and visualization, ensuring comprehensive skill coverage.

Project Goal: To analyse game review scores, platform popularity, and genre distribution over time using the `ign.csv` dataset.

The **primary libraries/modules** required for this project are:

- **Pandas:** For all data manipulation, filtering, aggregation, and feature engineering tasks.
- **NumPy:** Specifically for numerical calculations like yearly growth (`np.diff`).
- **Visualization Library (e.g., Matplotlib):** For generating all required plots (Line Graphs, Bar Charts, Pie Charts, Stacked Bar Charts).

Project Coverage

Category	Skills Covered
Data Pre-processing	Data Loading, Missing Value Handling (Mean/Mode), Type Conversion, Column Removal
Pandas Operations	Grouping, Filtering, Aggregation, Feature Engineering (score_category, editors_choice conversion)
NumPy Calculations	Array conversion, Yearly score growth calculation (np.diff)
Visualization	Line Chart, Bar Chart, Pie Chart, Histogram, Stacked Bar Chart, Saving Plots

Scenario 1: Basic Data Loading & Cleaning (□ Easy)

Description:
In this scenario we are loading the dataset ign.csv to the DataFrame. After that we are displaying the first and last 5 rows along with shape of the dataset. And delete the unnecessary column from the dataset. Missing values from score column is replaced with mean value and genre column with the mode. We need to ensure the values have the proper datatypes.

Task	Description
1.	Load the `ign.csv` dataset using Pandas.
2.	Display the first 5 rows (`head()`), last 5 rows (`tail()`), and the shape of the dataset.
3.	Remove the unnecessary column: `"Unnamed: 0"`.
4.	Check for missing values in `score`, `genre`, and `platform`.
5.	Handle missing values: Fill the numeric column `score` with the mean, and the categorical column `genre` with the mode.
6.	Ensure correct data types: Convert `score` → float and `release_year`, `release_month`, `release_day` → integer.

Task Workflow:

The dataset is uploaded to the DataFrame using **read_csv()** function. Once the dataset is loaded we can check the top 5 rows and bottom 5 rows using **head()** and **tail()** functions respectively. To check the shape of the dataset we can use **df.shape** attribute. The result is as shown below.

```
Top 5 rows from the dataset are
    Unnamed: 0 score_phrase  ... release_month release_day
0            0      Amazing  ...             9          12
1            1      Amazing  ...             9          12
2            2        Great  ...             9          12
3            3        Great  ...             9          11
4            4        Great  ...             9          11

[5 rows x 11 columns]
Last 5 rows from the dataset are
        Unnamed: 0 score_phrase  ... release_month release_day
18620       18620         Good  ...             6          29
18621       18621      Amazing  ...             6          29
18622       18622     Mediocre  ...             6          28
18623       18623  Masterpiece  ...             6          28
18624       18624  Masterpiece  ...             6          28

[5 rows x 11 columns]
Shape of Dataset is:  (18625, 11)
```

To remove the unnecessary columns which don't have any column names we can use **drop()** method. After using drop function, the table won't be having unnamed columns as shown below.

```
      score_phrase  ... release_day
0          Amazing  ...          12
1          Amazing  ...          12
2            Great  ...          12
3            Great  ...          11
4            Great  ...          11
...            ...  ...         ...
18620         Good  ...          29
18621      Amazing  ...          29
18622     Mediocre  ...          28
18623  Masterpiece  ...          28
18624  Masterpiece  ...          28

[18625 rows x 10 columns]
```

To check the missing values for more than one columns we can use the a list [ ] inside the selection ['column 1',' column 2',' column 3'] along with the **isna()** function. Also we can count the number of cells that don't have any value using **sum()** function along with isna().

Once we have the count we are replacing the missing values of score column with mean value and genre column with mode value (frequent value of that column) using **fillna()** function. The output is as shown below.

```
The total number of missing values are
 score        0
genre       36
platform     0
dtype: int64
After updating missing values count are
 score       0
genre       0
platform    0
dtype: int64
```

To change the datatypes of selected columns we use **astype()** function. For a single column we can directly specify the column name and use astype(). For more than one columns we can use dictionary or list and provide the column names. After conversion the datatypes of columns are as shown below.

```
<class 'pandas.DataFrame'>
RangeIndex: 18625 entries, 0 to 18624
Data columns (total 10 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   score_phrase    18625 non-null  str
 1   title           18625 non-null  str
 2   url             18625 non-null  str
 3   platform        18625 non-null  str
 4   score           18625 non-null  float64
 5   genre           18625 non-null  str
 6   editors_choice  18625 non-null  str
 7   release_year    18625 non-null  int64
 8   release_month   18625 non-null  int64
 9   release_day     18625 non-null  int64
dtypes: float64(1), int64(3), str(6)
memory usage: 1.4 MB
```

Conclusion:

In this scenario we loaded the dataset to the pandas DataFrame. Displayed the top and bottom columns. Cleaned the dataset by removing the unwanted columns and updated the missing cells with proposed values. And also the datatypes are updated as per the requirement, and the dataset is now more accurate and ready for further analysis.

Scenario 2: Line Graph (Score Trend) + Save (□ Easy)

Description:
In this scenario we are grouping the data by a release year column and calculating the average score. Once the average is converted into numpy array we are plotting the line graph for deeper analysis. And we are storing the graph in graph folder for future reference.

Task	Description
1.	Group data by `release_year`.
2.	Calculate the average score per year using Pandas.
3.	Convert the resulting average scores and years into NumPy arrays.
4.	Plot a line graph with `release_year` on the X-axis and average score on the Y-axis.
5.	Add the title: "Average Game Score Over Years" and proper axis labels.
6.	Save the graph: `plt.savefig("avg_score_trend.png")`.

Task Workflow:
As per the requirement we are grouping the data by the column release_year using groupby() function. And in the next step calculating the average score using mean() function. After calculating average the next step is to convert the results in to numpy array. For that we are using to_numpy() function, which is faster and reliable than np.array(). After these operations the result is as shown below.

```
release_year
1970    8.500000
1996    6.161745
1997    6.566341
1998    6.930678
1999    6.983484
2000    6.693589
```

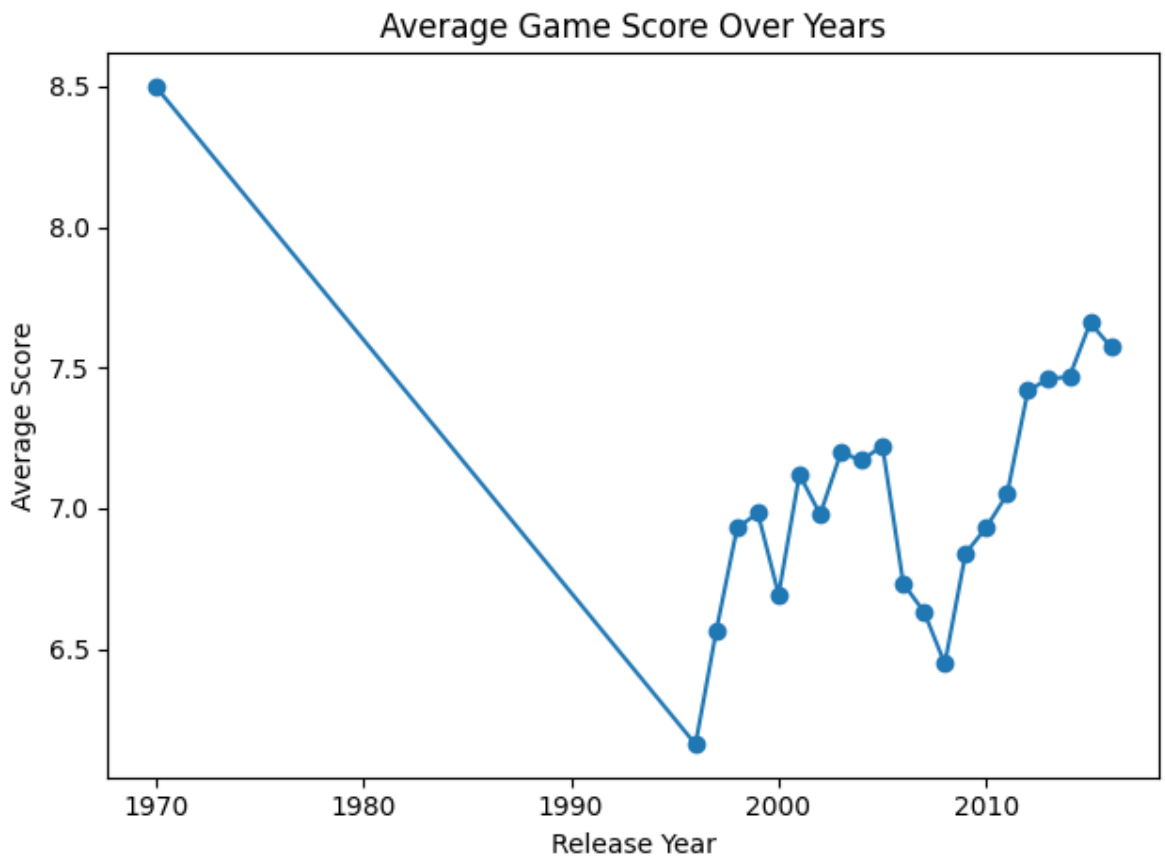
Note: This is a cropped image, in the output we can see all years data.

The output of results converted to array is as shown below.

```
Years Array:
 [1970 1996 1997 1998 1999 2000 2001 2002 2003 2004 2005 2006 2007 2008
 2009 2010 2011 2012 2013 2014 2015 2016]
Scores Array:
 [8.5        6.16174497 6.56634146 6.93067847 6.98348387 6.69358852
 7.12049419 6.97852564 7.19824561 7.17071214 7.22242647 6.73286424
 6.63254658 6.44814621 6.84036752 6.93286867 7.05032619 7.41832669
 7.46048851 7.46759777 7.65945205 7.57345133]
```

Graph:

In this step we are providing the visual representation of the score and release year data for better understanding. **plot()** function is used to plot the graph.



Conclusion:

In this scenario we grouped the data by the release year and calculated the average score per year. And the data is stored in numpy array. With the help of plot() function line graph is created for better understanding of average score over the release years. And the chart is stored for future references.

Scenario 3: Filtering + Bar Chart + Save (□ Medium)

Description:
In this scenario we need to filter the dataset with the condition score column having values greater than 7. And count the number of high rated games per platform. After that select top 10 platforms using pandas and plot the bar graph for visual representation.

Task	Description
1.	Filter the dataset to include only games where `score > 7`.
2.	Count the number of high-rated games per platform.
3.	Select the top 10 platforms using Pandas.
4.	Convert the platform counts into NumPy arrays.
5.	Plot a bar chart with `platform` on the X-axis and count of games on the Y-axis.
6.	Rotate x-axis labels for readability.
7.	Save the graph: `plt.savefig("top_platforms_bar.png")`.

Task Workflow:
To filter the score column with value greater than 7, we can simply write the condition along with the column name. After filtering, the dataset looks as shown below.

```
      score_phrase  ... release_day
0          Amazing  ...          12
1          Amazing  ...          12
2            Great  ...          12
3            Great  ...          11
4            Great  ...          11
...            ...  ...         ...
18619         Good  ...          28
18620         Good  ...          29
18621      Amazing  ...          29
18623  Masterpiece  ...          28
18624  Masterpiece  ...          28

[9800 rows x 10 columns]
```

To count the number of high-rated games per platform we can use **value.counts()** function and store it in a separate variable. Squeezed text() have all the platform names along with the count. To print the top 10 platforms we are using **head()** function. The resulted columns are as shown below.

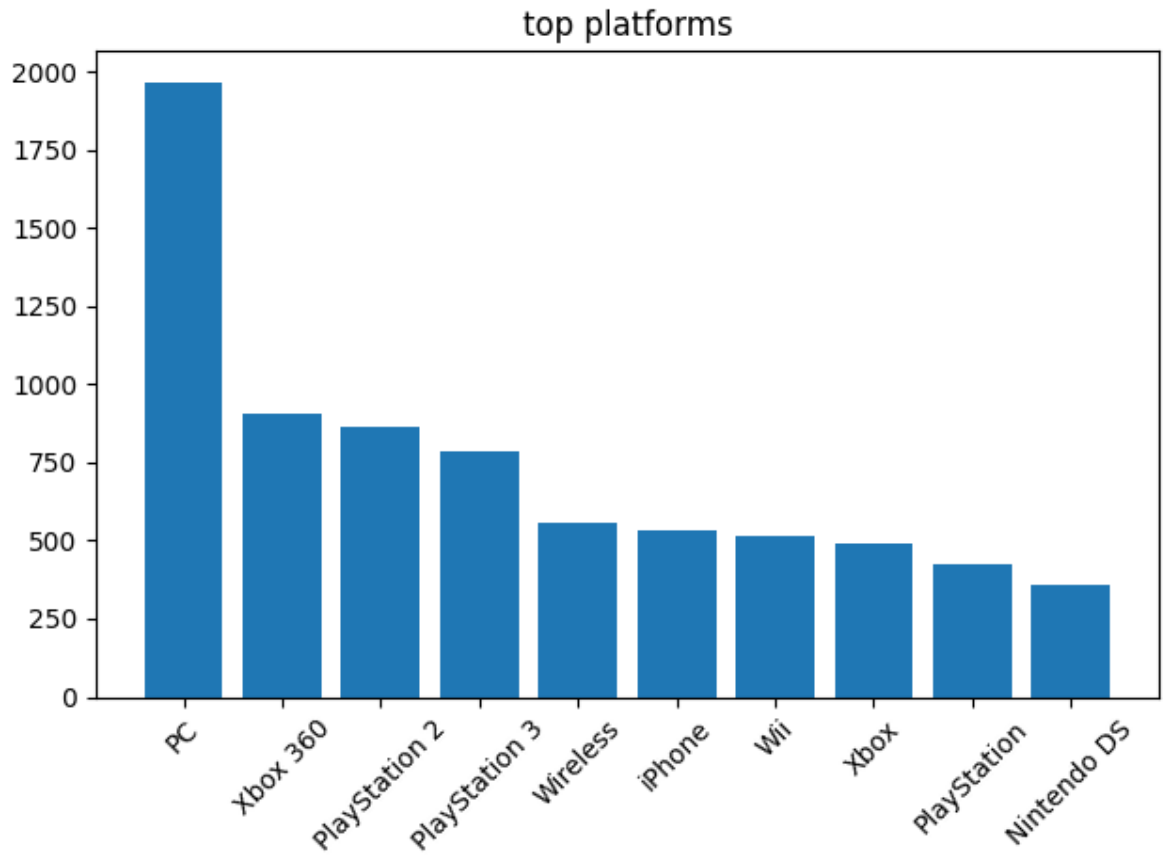
```
Squeezed text (56 lines).
platform
PC                 1968
Xbox 360            909
PlayStation 2       864
PlayStation 3       783
Wireless            554
iPhone              531
Wii                 512
Xbox                489
PlayStation         424
Nintendo DS         356
Name: count, dtype: int64
```

In the next step we can convert the data into numpy array using **to_numpy()** function and the result is as shown below.

```
Platforms: ['PC' 'Xbox 360' 'PlayStation 2' 'PlayStation 3' 'Wireless' 'iPhone' 'Wii'
 'Xbox' 'PlayStation' 'Nintendo DS']
Counts: [1968  909  864  783  554  531  512  489  424  356]
```

Graph:

This graph is the graphical representation of top platforms with greater than 7 rating and number of games played on those platforms.



Conclusion:

In this scenario we filtered the dataset with a column having rating greater than 7. With that resulted dataset we are able to identify the game counts and platforms which have better ratings. After storing the data in numpy array, we used bar chart to provide the comparison between top 10 gaming platforms with better rating. And the graph is stored in Graphs folder for future references.

Scenario 4: Aggregation + Pie Chart + Save (□ Medium)

Description:
In this scenario the task is to count the number of games per genre. After that select the top 5 genres using pandas and visualize the data using pie chart. And save the chart for future references.

Task	Description
1.	Count the total number of games per `genre`.
2.	Select the top 5 genres using Pandas.
3.	Prepare labels (genres) and values (counts) for the pie chart.
4.	Plot a pie chart using `genre` as labels and `count` as values.
5.	Add percentage labels using `autopct` (e.g., '%.1f%%').
6.	Save the graph: `plt.savefig("genre_distribution.png")`.

Task Workflow:
The first step is count the number of games per genre. To achieve that we can use **value_counts()** function on genre column. This will give the output as follows.

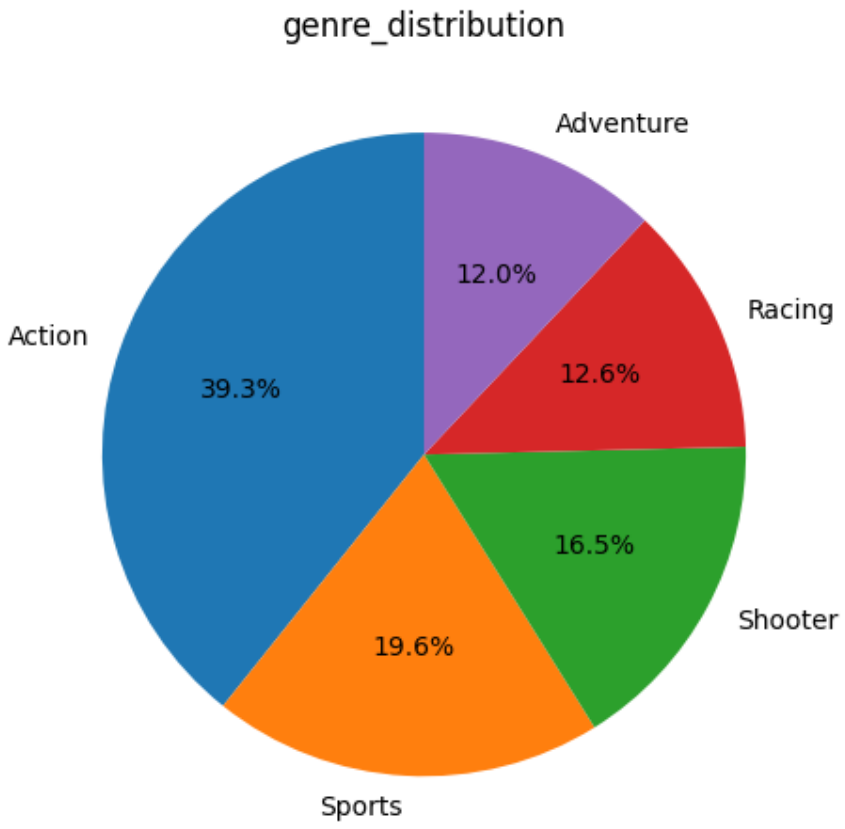
```
genre
Action                    3833
Sports                    1916
Shooter                   1610
Racing                    1228
Adventure                 1175
                          ...
Action, Strategy             1
Puzzle, Platformer           1
Music, Adventure             1
Sports, Golf                 1
Shooter, Adventure           1
Name: count, Length: 112, dtype: int64
```

After that we can count the top 5 games using **head()** function. The top 5 games are shown below.

```
genre
Action        3833
Sports        1916
Shooter       1610
Racing        1228
Adventure     1175
Name: count, dtype: int64
```

Graph:

To get the pictorial representation of top 5 games contribution we are using pie chart along with the **autopct** attribute to show percentage as shown below.



Conclusion:

In this scenario we checked for the total number of games per genre using value_counts() function. Then we selected the top 5 games from the column along with the count to provide graphical representation using pie chart. The autopct attribute is used to show the percentages as well. And the graph is stored in the given path for future references.

Scenario 5: Advanced Analysis + Multiple Graphs (□ Hard)

Description:
In this scenario, create a column to add comments for each game based on users rating. Convert the editor's choice to 0 and 1. Calculate the difference for score growth on yearly scores. Finally visualize the dataset and store the graphs in a folder. And provide the insights about the data analysed.

Task	Description
1.	Create a new column `score_category`: "Excellent" for score ≥ 9, "Good" for $7 \le$ score < 9, and "Average" for score < 7.
2.	Convert the `editors_choice` column: 'Y' $\rightarrow 1$, 'N' $\rightarrow 0$.
3.	Use NumPy's `np.diff()` to calculate the **yearly score growth** based on the average yearly scores from Scenario 2.
4.	Plot a **Line Graph** showing the trend of average score per `release_year`.
5.	Plot a **Stacked Bar Chart** showing the count of `score_category` per `release_year`.
6.	Plot a **Histogram** showing the distribution of `score`.
7.	Save the graphs: `plt.savefig("score_trend.png")`, `plt.savefig("score_category_stacked.png")`, `plt.savefig("score_distribution.png")`.
8.	Identify: Which years had the highest scores, whether high scores increased over time, and if `editors_choice` correlates with high scores.

Task Workflow:

In the first task we need to create a new column to add comments to the games as per user rating. For that we are using **df.loc[]** property to add the comments along with the condition. And also we are changing the editor's choice to 0 and 1 using **replace()** function. The updated dataset is as shown below.

```
0        Excellent
1        Excellent
2             Good
3             Good
4             Good
           ...
18620         Good
18621    Excellent
18622      Average
18623    Excellent
18624    Excellent
Name: score_category, Length: 18625, dtype: str
```

Fig1: Adding comments based on user ratings.

```
0        1
1        1
2        0
3        0
4        0
        ..
18620    0
18621    1
18622    0
18623    1
18624    1
Name: editors_choice, Length: 18625, dtype: object
```

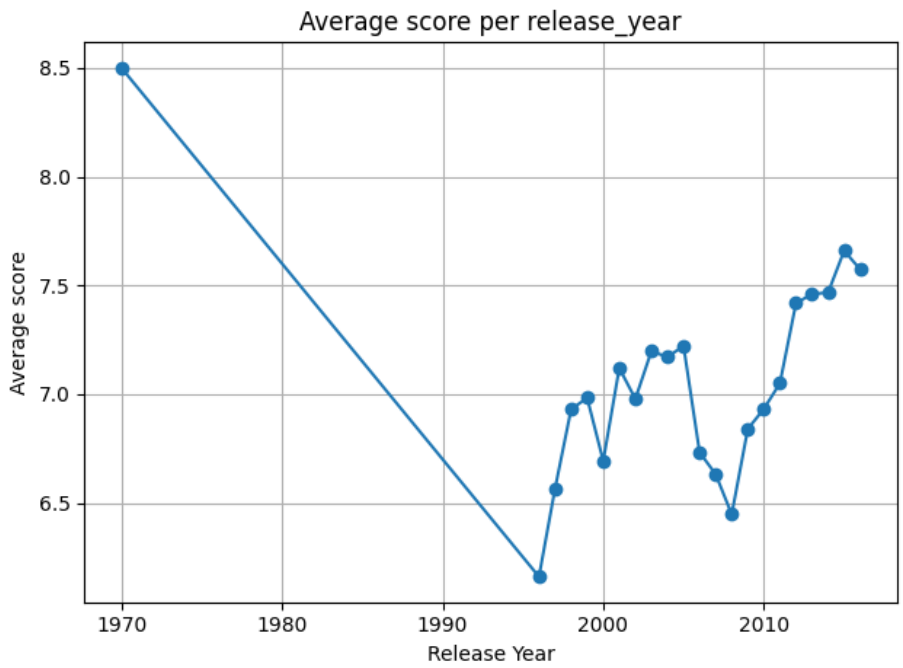
Fig2: Updated editors_choice column

In the next step we are calculating the difference between early score growth and average yearly scores using **np.diff()** function. Before that we need to have the yearly average, so we are using **groupby()** function with release year and score column along with **mean()** function to get the average. With the help of np.insert() function we can only store the year and the average score of each year in a variable and display the yearly growth as shown below.

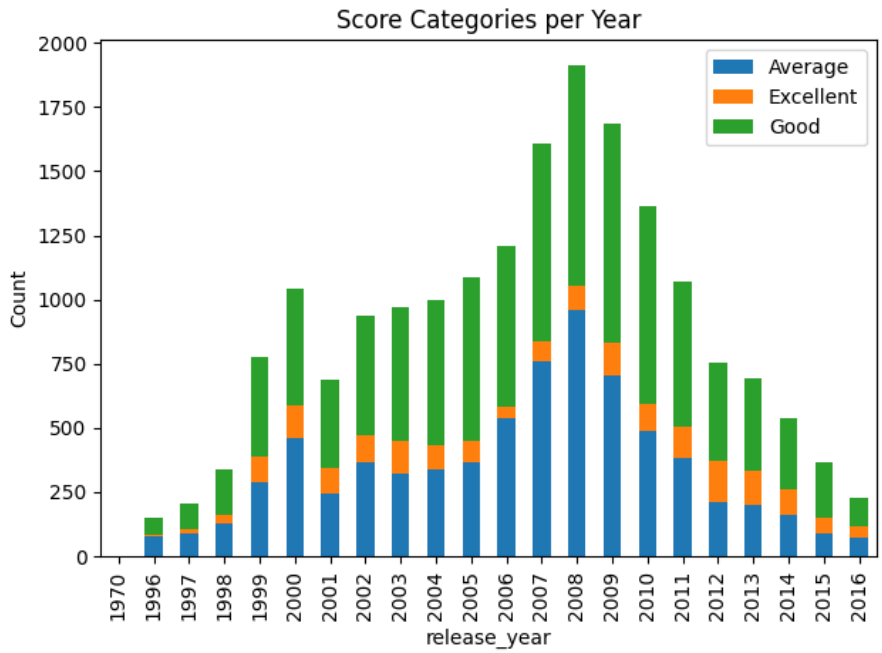
```
Yearly Growth
1970: Starting year (Avg Score: 8.50)
1996: -2.34
1997: +0.40
1998: +0.36
1999: +0.05
2000: -0.29
2001: +0.43
2002: -0.14
```

Graphs:

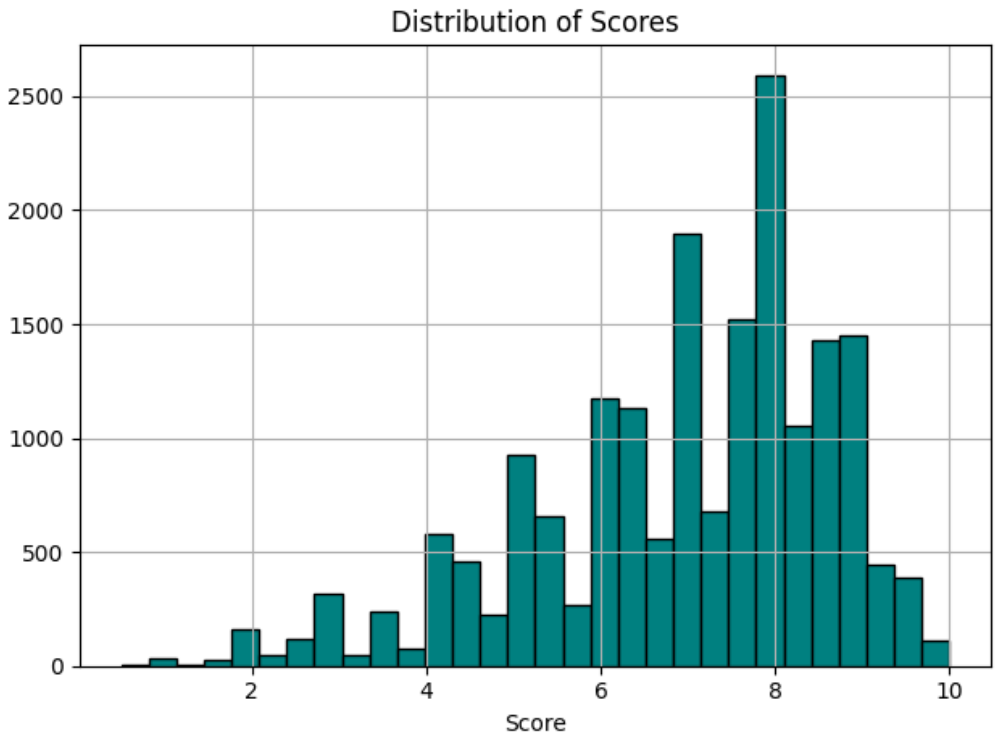
1. Line Graph: To have the graphical representation of the required data we can use matplotlib module. To average score per release year we are using Line Chart with the plot() function. Using title(), xlabel(), ylabel() and savefig() functions to add title, label to x axis, label to y axis and to save the graph respectively. The line graph is as shown below.



2. Stacked Bar Graph: To count the score category per release year we can use stacked bar graph. We are storing the release year and scoring category values in count variable and plotting the graph with **plot()** function along with **kind=bar** as argument. The resulted graph is as shown below.



3. Histogram: To obtain distribution of scores we are using hist() function along with the score column, and the resulted graph is as shown below.



Final step is to provide the insights of the analysis.

1. After analysing the dataset on various conditions, we can conclude that the Year with high score is 1970 and the score is 8.5.
2. After observing the results we can see that, in 1970, the average score was 8.50. and by 2016, it has been changed to 7.57. So we can conclude that, the average scores have not increased over time.
3. As we can see significance difference in editors_choice as 0 marked games has less voting and 1 marked games has more voting. Finally, we can say - Yes, Editor's Choice games correlate with higher scores.

Conclusion:

In this scenario we added a new column for adding comments based on user rating. And we calculated the difference between yearly score growth with average yearly score using np.diff() function. For graphical visualization of required data we used line graph, stacked bar chart and histogram for better understanding of the dataset. And also provided the insights based on the analysis.

Final Conclusion:

Depending on all the scenarios, operations and graphs, we can see there are major changes happened in gaming industry in recent years. As the games started to evolve, the users started experiencing many types of games on many platforms. Some games met user expectations got good ratings and some didn't that we can see in the table as well as on graphs. So we can conclude that the games with action attracts more users. The best platform to play games is PC. The year 2008 have the most user counts with mixed ratings overall. And we can see that in recent years' users have started to play more games.

What you have learned

In this project I learned about how to clean the data in pandas. Also learned how to group required data/columns and perform operations on those grouped data. And also how to filter the required data from a huge data set to operate on them. Then how to aggregate the data to provide the expected result. And as a part of numpy, I learned how to get the difference of particular columns and how to create an array from a dataset with multiple columns. As a part of visualisation I learned how to plot different kind of graphs, which function is used for which kind of plots and how to save them in required path for future references.